

Dialectos CSV en Python

Guía completa de dialect, delimiter, quotechar, quoting, doublequote, escapechar, skipinitialspace, lineterminator y strict — con ejemplos, resultados y cuándo usar cada uno.

El módulo csv de Python usa "dialectos" para definir cómo se interpretan y escriben los archivos CSV. Un dialecto es un conjunto de parámetros que describen el formato: qué carácter separa los campos, cómo se protegen los valores especiales, etc. Puedes usar los dialectos predefinidos de Python o crear los tuyos propios.

1. delimiter

carácter separador

Define qué carácter separa los campos (columnas) dentro de cada fila. Es el parámetro más importante y visible de todos.

delimiter

str (1 carácter)

default: ','

Cada campo en una fila está separado por este carácter. Si tu dato contiene ese carácter, el campo debe entrecomillarse. No puede ser un salto de línea ni \r.

EJEMPLO — COMA (POR DEFECTO)

```
import csv, io
data = [['Ana', '28', 'Madrid'], ['Luis', '35', 'Sevilla']]
out = io.StringIO()
writer = csv.writer(out, delimiter=',')
writer.writerows(data)
print(out.getvalue())
```

RESULTADO

```
Ana,28,Madrid
Luis,35,Sevilla
```

EJEMPLO — PUNTO Y COMA (MUY COMÚN EN EUROPA)

```
writer = csv.writer(out, delimiter=';')
```

RESULTADO

```
Ana;28;Madrid
Luis;35;Sevilla
```

Excel en España suele generar archivos con ; como separador porque la coma se usa como separador decimal (1,5 en vez de 1.5).

EJEMPLO — TABULADOR (TSV)

```
writer = csv.writer(out, delimiter='\t')
# Útil cuando los datos contienen comas y puntos y comas
```

RESULTADO

```
Ana 28 Madrid
Luis 35 Sevilla
```

EJEMPLO — DATO QUE CONTIENE EL DELIMITADOR

```
data = [['García, Ana', '28', 'Madrid']]
writer = csv.writer(out, delimiter=',')
# Python entrecomilla automáticamente el campo con coma
```

RESULTADO

```
"García, Ana",28,Madrid
```

Python gestiona esto automáticamente — no tienes que preocuparte si usas `csv.writer`.

,

Estándar internacional

;

Excel europeo

\t

TSV / datos con comas

|

Logs, exportaciones

:

Datos de sistema

Usar ; cuando...

- El archivo viene de Excel en España/Europa
- Los datos contienen muchas comas
- Es un archivo financiero con decimales con coma

Cuidado con | o :

- No están estandarizados
- Muchas herramientas no los soportan
- Úsalos solo si controlas ambos extremos

2. quotechar

carácter de cita

Define qué carácter se usa para "envolver" campos que contienen caracteres especiales (el delimitador, saltos de línea, etc.).

quotechar

str (1 carácter)

default: '''

Cuando un campo contiene el delimitador o un salto de línea, se envuelve con `quotechar`. Puede ser `None` si usas `quoting=QUOTE_NONE`.

EJEMPLO — COMILLA DOBLE (POR DEFECTO)

```
data = ["Hola, mundo", "Normal", "Otro campo"]
csv.writer(out, delimiter=",", quotechar='"')
# El primer campo tiene coma → se entrecomilla
```

RESULTADO

```
"Hola, mundo",Normal,Otro campo
```

EJEMPLO — COMILLA SIMPLE COMO QUOTECHAR

```
csv.writer(out, delimiter=',', quotechar="'')
```

RESULTADO

```
'Hola, mundo',Normal,Otro campo
```

Cambiar `quotechar` a `'` puede causar problemas con muchas herramientas que esperan `"`. Úsalo solo si el sistema receptor lo requiere.

EJEMPLO — CAMPO CON SALTO DE LÍNEA

```
data = ["Descripción\nlarga", "Precio"]
csv.writer(out, delimiter=",")
```

RESULTADO

```
"Descripción
larga",Precio
```

Python gestiona los saltos de línea dentro de campos envolviéndolos en comillas automáticamente.

3. quoting

comportamiento de citas

Controla en qué situaciones se añaden comillas alrededor de los campos. Es el parámetro más complejo e importante después de delimiter.

quoting

constante csv.QUOTE_*

default: csv.QUOTE_MINIMAL

QUOTE_MINIMAL — SOLO CUANDO ES NECESARIO (POR DEFECTO)

```
data = ["Ana", "Hola, mundo", "42"]
csv.writer(out, quoting=csv.QUOTE_MINIMAL)
```

RESULTADO

```
Ana,"Hola, mundo",42
```

Solo se cita el campo que contiene la coma. Los demás se dejan sin comillas. Usa esto para la mayoría de casos.

QUOTE_ALL — CITA TODOS LOS CAMPOS SIEMPRE

```
csv.writer(out, quoting=csv.QUOTE_ALL)
```

RESULTADO

```
"Ana", "Hola, mundo", "42"
```

Útil cuando la herramienta destino espera comillas en todo, o cuando quieres dejar claro que todos los valores son texto.

QUOTE_NONNUMERIC — CITA TODO EXCEPTO NÚMEROS

```
data = ["Ana", "Hola, mundo", 42] # 42 es int, no string
csv.writer(out, quoting=csv.QUOTE_NONNUMERIC)
```

RESULTADO

```
"Ana", "Hola, mundo", 42
```

Al leer con QUOTE_NONNUMERIC, todos los campos sin comillas se convierten automáticamente a float. Muy útil para CSVs numéricos.

QUOTE_NONE — NO CITA NUNCA (PELIGROSO)

```
import csv
csv.writer(out, quoting=csv.QUOTE_NONE, escapechar='\\')
```

RESULTADO

```
Ana,Hola\, mundo,42
```

Si un campo contiene el delimitador y no hay escapechar, Python lanzará una excepción. Siempre define escapechar junto a QUOTE_NONE.

QUOTE_STRINGS — (PYTHON 3.12+) CITA SOLO STRINGS

```
# Solo disponible desde Python 3.12
csv.writer(out, quoting=csv.QUOTE_STRINGS)
```

RESULTADO

```
"Ana", "Hola, mundo", 42
```

Similar a `QUOTE_NONNUMERIC` pero más semántico: cita cualquier valor de tipo `str`, sin importar si parece número.

4. `doublequote`

escape de comillas

Define cómo se escapan las comillas que aparecen dentro de un campo ya entrecomillado. La estrategia por defecto es duplicarlas.

`doublequote`

`bool`

`default: True`

Cuando un campo contiene el carácter quotechar (por defecto `"`) y `doublequote=True`, Python lo escapa duplicándolo: `""`. Si es `False`, usa escapechar en su lugar.

DOUBLEQUOTE=TRUE (POR DEFECTO) — LA COMILLA SE DUPLICA

```
data = [{"Él dijo \"hola\"", "Normal"}]
csv.writer(out, doublequote=True)
```

RESULTADO

```
"Él dijo ""hola""",Normal
```

Al leer de vuelta, `""` se interpreta como una sola `"`. Es el estándar RFC 4180.

DOUBLEQUOTE=FALSE — USA ESCAPECHAR EN VEZ DE DUPLICAR

```
csv.writer(out, doublequote=False, escapechar='\\')
```

RESULTADO

```
"Él dijo \"hola\"",Normal
```

Menos estándar. Úsalo si el sistema receptor espera backslash-escape (como algunos exportadores de bases de datos MySQL).

5. escapechar

carácter de escape

Carácter alternativo para escapar caracteres especiales. Se usa cuando `doublequote=False` o con `QUOTE_NONE`.

escapechar

str (1 carácter) o None

default: None

Por defecto es None (no se usa). Solo es necesario cuando `doublequote=False` o `quoting=QUOTE_NONE`. El carácter más común es `\`.

EJEMPLO CON QUOTE_NONE + ESCAPECHAR

```
data = [['Ana', 'Madrid, España', '42']]
csv.writer(out, quoting=csv.QUOTE_NONE, escapechar='\\')
```

RESULTADO

```
Ana,Madrid\, España,42
```

El backslash "protege" la coma dentro del campo para que no se confunda con el delimitador.

SIN ESCAPECHAR + QUOTE_NONE → EXCEPCIÓN

```
csv.writer(out, quoting=csv.QUOTE_NONE)
# Si un campo contiene el delimitador:
# Error: csv.Error: need to escape, but no escapechar set
```

Nunca uses `QUOTE_NONE` sin definir `escapechar` si tus datos pueden contener el delimitador.

6. skipinitialspace

espacios tras delimitador

Controla si se ignora el espacio en blanco que hay justo después del delimitador al leer un archivo.

skipinitialspace

bool

default: False

Muy útil cuando recibes archivos CSV donde pusieron un espacio después de cada coma por legibilidad: Ana, 28, Madrid. Sin esta opción, el "28" llegaría como " 28".

SIN SKIPINITIALSPACE (POR DEFECTO) — EL ESPACIO FORMA PARTE DEL DATO

```
import csv, io
texto = "Ana, 28, Madrid"
reader = csv.reader(io.StringIO(texto), skipinitialspace=False)
print(list(reader))
```

RESULTADO

```
[['Ana', ' 28', ' Madrid']]
```

CON SKIPINITIALSPACE=TRUE — EL ESPACIO SE DESCARTA

```
reader = csv.reader(io.StringIO(texto), skipinitialspace=True)
print(list(reader))
```

RESULTADO

```
[['Ana', '28', 'Madrid']]
```

Úsalo siempre que el CSV tenga espacios decorativos después de los delimitadores. Solo afecta a la lectura, no a la escritura.

7. lineterminator

fin de línea

Define qué secuencia de caracteres se usa para terminar cada fila al escribir. Afecta solo a la escritura.

lineterminator

str

default: '\r\n'

La terminación de línea por defecto es \r\n (CRLF), que es el estándar RFC 4180 y lo que espera Windows/Excel. En sistemas Unix/Linux/macOS moderno se usa \n.

\r\n

CRLF — Windows/Excel (default)

\n

LF — Unix/Linux/macOS

\r

CR — macOS antiguo (raro)

DIFERENCIA EN BYTES (REPRESENTACIÓN VISUAL)

```
# Con \r\n (default):
"Ana,28,Madrid\r\nLuis,35,Sevilla\r\n"

# Con \n:
"Ana,28,Madrid\nLuis,35,Sevilla\n"
```

Cuando abres un archivo con `open(..., newline="")` (recomendado por Python para CSV), el módulo `csv` gestiona los saltos de línea por ti. Así evitas duplicados como `\r\n`.

PRÁCTICA RECOMENDADA AL ESCRIBIR ARCHIVOS CSV

```
with open('archivo.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.writer(f) # usa \r\n por defecto (correcto)
    writer.writerow(['Ana', '28', 'Madrid'])
```

8. strict

validación estricta

Activa el modo estricto: lanza una excepción si el CSV tiene un formato incorrecto en lugar de intentar parsear de todas formas.

strict

bool

default: False

Por defecto Python intenta ser tolerante con CSV malformados. Con `strict=True`, cualquier violación del formato lanza `csv.Error` inmediatamente.

STRICT=FALSE (POR DEFECTO) — TOLERANTE CON ERRORES

```
# CSV con comillas mal cerradas
texto = 'Ana,"campo mal abierto,28'
reader = csv.reader(io.StringIO(texto), strict=False)
print(list(reader))

# Intenta parsear igualmente, resultado impredecible
```

RESULTADO (COMPORTAMIENTO IMPREDECIBLE)

```
[['Ana', 'campo mal abierto,28']]
```

STRICT=TRUE — FALLA EXPLÍCITAMENTE

```
reader = csv.reader(io.StringIO(texto), strict=True)
list(reader)
# Lanza: csv.Error: ',' expected after ''
```

Usa `strict=True` cuando necesites garantizar la integridad del CSV (pipelines de datos, validación de imports). Deja `False` en prototipos o cuando aceptes CSVs de fuentes externas poco confiables.

9. Dialectos built-in de Python

incluidos

Python incluye 3 dialectos predefinidos que puedes usar directamente. También puedes registrar los tuyos propios.

Dialecto	delimiter	quotechar	doublequote	lineterminator	quoting	skipinitialspace
excel (default)	,	"	True	\r\n	MINIMAL	False
excel-tab	\t	"	True	\r\n	MINIMAL	False
unix	,	"	True	\n	ALL	False

USAR UN DIALECTO POR NOMBRE

```
csv.writer(f, dialect='excel') # por defecto
csv.writer(f, dialect='excel-tab') # para TSV
csv.writer(f, dialect='unix') # para sistemas Unix
```

REGISTRAR UN DIALECTO PERSONALIZADO

```
csv.register_dialect(
    'mi_dialecto',
    delimiter=';',
    quotechar='"',
    doublequote=True,
    skipinitialspace=True,
    lineterminator='\n',
    quoting=csv.QUOTE_MINIMAL
)
# Ahora puedes reutilizarlo en todo tu proyecto:
csv.writer(f, dialect='mi_dialecto')
```

```
csv.reader(f, dialect='mi_dialecto')
```

Registrar dialectos es muy útil en proyectos donde lees/escribes muchos archivos del mismo formato. Define el dialecto una vez, úsalo en todos lados.

10. Combinaciones habituales

casos reales

Estos son los patrones más frecuentes en proyectos reales de Python.

CSV estándar internacional

Para compartir entre herramientas y países anglosajones.

```
csv.writer(f,  
delimiter=',',  
quoting=csv.QUOTE_MINIMAL)
```

CSV para Excel España

Archivo que se abre bien en Excel con localización española.

```
csv.writer(f,  
delimiter=';',  
quotechar='"',  
doublequote=True)
```

TSV para datos con comas

Si tus datos tienen muchas comas, el tabulador evita conflictos.

```
csv.writer(f,  
dialect='excel-tab')
```

CSV de máxima seguridad

Todo entrecomillado, validación estricta, para pipelines críticos.

```
csv.writer(f,  
quoting=csv.QUOTE_ALL,  
strict=True)
```

CSV numérico

Los números no llevan comillas; el lector los convierte a float.

```
csv.reader(f,  
quoting=csv.QUOTE_NONNUMERIC)
```

CSV con escapechar (MySQL)

Exportaciones de MySQL que usan backslash en vez de doublequote.

```
csv.reader(f,  
doublequote=False,  
escapechar='\\')
```

11. ¿Lectura, escritura o ambos?

referencia rápida

Cada parámetro del dialecto no afecta igual a `csv.reader` y `csv.writer`. Esta sección aclara exactamente en qué sentido actúa cada uno.

Parámetro	Lectura (reader)	Escritura (writer)
delimiter	Identifica dónde termina cada campo al parsear la línea.	Inserta este carácter entre cada campo al generar la línea.
quotechar	Reconoce qué carácter envuelve los campos especiales y lo elimina del valor final.	Inserta este carácter alrededor de los campos que lo necesiten.
quoting	QUOTE_NONNUMERIC convierte a float los campos sin comillas. Los demás modos no cambian el comportamiento de lectura.	Decide cuándo añadir comillas: siempre, nunca, solo si es necesario, solo si no es número.
doublequote	Interpreta "" dentro de un campo como una sola " al extraer el valor.	Cuando un campo contiene quotechar, lo duplica (") en vez de usar escapechar.
escapechar	Desescapa el carácter que sigue al escapechar al leer el valor.	Usa este carácter para proteger el quotechar o el delimiter cuando doublequote=False o QUOTE_NONE.
skipinitialspace	Descarta el espacio inmediatamente después del delimiter al extraer el campo.	Sin efecto. El writer nunca añade espacios tras el delimiter.
lineterminator	Sin efecto directo. El reader detecta <code>\r\n</code> , <code>\n</code> y <code>\r</code> automáticamente.	Usa esta secuencia para terminar cada fila al escribir.
strict	Con True lanza <code>csv.Error</code> ante cualquier formato inválido en lugar de intentar recuperarse.	Efecto mínimo en escritura; el writer raramente genera CSVs malformados por sí solo.

Solo escritura: `lineterminator` — el reader ignora este parámetro y detecta los saltos de línea automáticamente (`\r\n`, `\n`, `\r`).

Solo lectura (en la práctica): `skipinitialspace` — el writer nunca inserta espacios tras el delimiter, así que este parámetro no tiene efecto al escribir.

Regla de oro: Si defines un dialecto personalizado para leer un archivo, usa exactamente los mismos parámetros al escribirlo. Si mezclas configuraciones entre reader y writer del mismo archivo, puedes escribir correctamente pero no leer el resultado después.